Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Curso: Bases de Datos I

Catedrático: Ing. Edgar René Ornelis Hoil

AutoAudit

Sistema de auditoría automatizado

Documentación técnica del proyecto final

Integrantes

Nombre completo	Carné
Kevin Josué Orellana Arbizú	0907-23-12047
Carlo René Hermógenes Rivera Estrada	0905-24-7010
Kevyn Eduardo Sarat Castrillo	3590-23-18725
William Daniel Salguero Ardon	6190-21-7287

Guatemala, 3 de noviembre de 2026

Índice

1. Introducción 4

2. Objetivos 4

2.1 Objetivo general 4

2.2 Objetivos específicos 4

3. Arquitectura y tecnologías 4

3.1 Flujo de funcionamiento 5

4. Modelo entidad-relación 6

4.1 Relaciones entre entidades 6

5. Diccionario de datos 8

5.1 EVENTO 8

5.2 TABLA_AUDITADA 8

5.3 COLUMNA_AUDITADA 9

5.4 CONFIGURACION_AUDITORIA 10

5.5 USUARIO_AUDITADO 10

5.6 ESTACION_TRABAJO 10

5.7 BITACORA_ENCABEZADO 11

5.8 BITACORA_DETALLE 11

5.9 LOG_GENERACION 12

5.10 PARAMETRO 12

5.11 Vistas 12

5.12 Funciones 13

6. Generación dinámica de triggers 15

6.1 Ejemplo de código generado (tabla tienda.productos) 15

7. Manual de configuración y activación de auditoría 18

7.1 Desde la interfaz web 18

7.2 Desde SQL 18
7.3 Estados de una tabla 18
8. Procedimiento de instalación y despliegue 20
8.1 Requisitos 20
8.2 Instalación en Windows 20
8.3 Instalación en Ubuntu 26.04 LTS 20
8.4 Orden de los scripts de base de datos 20
8.5 Usuarios de demostración 20
9. Consideraciones de seguridad 21
10. Pruebas de validación 22
10.1 Casos de prueba 22
10.2 Rendimiento con volumen creciente de registros 23
10.3 Tiempo de consultas con la bitácora poblada 24
11. Conclusiones 24
12. Referencias 25

1. Introducción

El presente documento describe el diseño, la implementación, la instalación y la validación de **AutoAudit**, un sistema de auditoría configurable para bases de datos relacionales en PostgreSQL. El sistema permite seleccionar, a partir del diccionario de datos del DBMS, qué tablas y columnas deben ser monitoreadas y qué operaciones registrar en cada una. Con esa configuración, AutoAudit genera automáticamente los triggers necesarios para registrar cada modificación en una bitácora centralizada, de la cual es posible consultar el historial de cambios por usuario o por columna específica.

La solución se compone de la base de datos con su lógica en PL/pgSQL, un servidor de aplicación en Python y una interfaz web para administrar la configuración, consultar la bitácora y exportar reportes.

El código fuente completo se encuentra en el repositorio: **https://github.com/KevinOrellana07/PROYECTOFINALBDD**

2. Objetivos

2.1 Objetivo general

Desarrollar una solución de auditoría automática que capture cambios en tablas configurables mediante la generación dinámica de triggers, permitiendo la consulta histórica de modificaciones por usuario o por columna específica.

2.2 Objetivos específicos

1. Diseñar el modelo entidad-relación completo del sistema de auditoría.
2. Implementar mecanismos de configuración para seleccionar tablas y columnas a auditar, con base en la consulta del diccionario de datos.
3. Desarrollar un generador automático de triggers basado en la configuración.
4. Crear una bitácora centralizada que registre todas las modificaciones.
5. Implementar consultas para el análisis histórico de cambios.

3. Arquitectura y tecnologías

El sistema se organiza en cuatro módulos que corresponden al alcance funcional del proyecto:

- **Configuración de auditoría:** consulta el diccionario de datos, permite seleccionar tablas, columnas y eventos, y valida su existencia antes de activar la auditoría.
- **Generación de triggers:** construye y ejecuta dinámicamente una función y un trigger *AFTER* exclusivos para cada tabla configurada, y gestiona las dependencias cuando la estructura de las tablas cambia.
- **Almacenamiento de bitácora:** registra cada cambio con su valor anterior y nuevo, el usuario responsable, la fecha y hora, la estación de trabajo y la dirección IP.
- **Consulta y reporte:** historial por usuario, por columna y por registro, con exportación a CSV, Excel y PDF.

Componente	Tecnología	Ubicación en el repositorio
Base de datos y lógica de auditoría	PostgreSQL 18, PL/pgSQL	sql/
Servidor de aplicación (API)	Python 3, FastAPI, psycopg 3	app/main.py
Interfaz web	HTML, CSS y JavaScript	app/static/
Reportes	CSV, Excel (openpyxl), PDF (reportlab)	app/main.py
Pruebas de validación	Scripts SQL con verificaciones	pruebas/
Instaladores	Ubuntu 26.04 LTS y Windows	install.sh, instalar_windows.bat

3.1 Flujo de funcionamiento

1. El administrador consulta el diccionario de datos y elige tablas, columnas y eventos.
2. La configuración se guarda en las tablas TABLA_AUDITADA, COLUMNA_AUDITADA y CONFIGURACION_AUDITORIA, validando cada elemento contra el catálogo de PostgreSQL.
3. El generador construye el código del trigger con las columnas y eventos configurados y lo ejecuta.
4. Cada INSERT, UPDATE o DELETE sobre la tabla dispara el trigger, que registra un encabezado por operación y un detalle por cada columna afectada.
5. Los auditores consultan el historial y exportan reportes desde la interfaz web.

4. Modelo entidad-relación

El modelo separa la configuración de la auditoría (qué se audita) de la bitácora (qué ocurrió). La bitácora está normalizada en encabezado y detalle, y hace referencia al usuario, la estación de trabajo, el evento y la configuración utilizada en cada cambio.

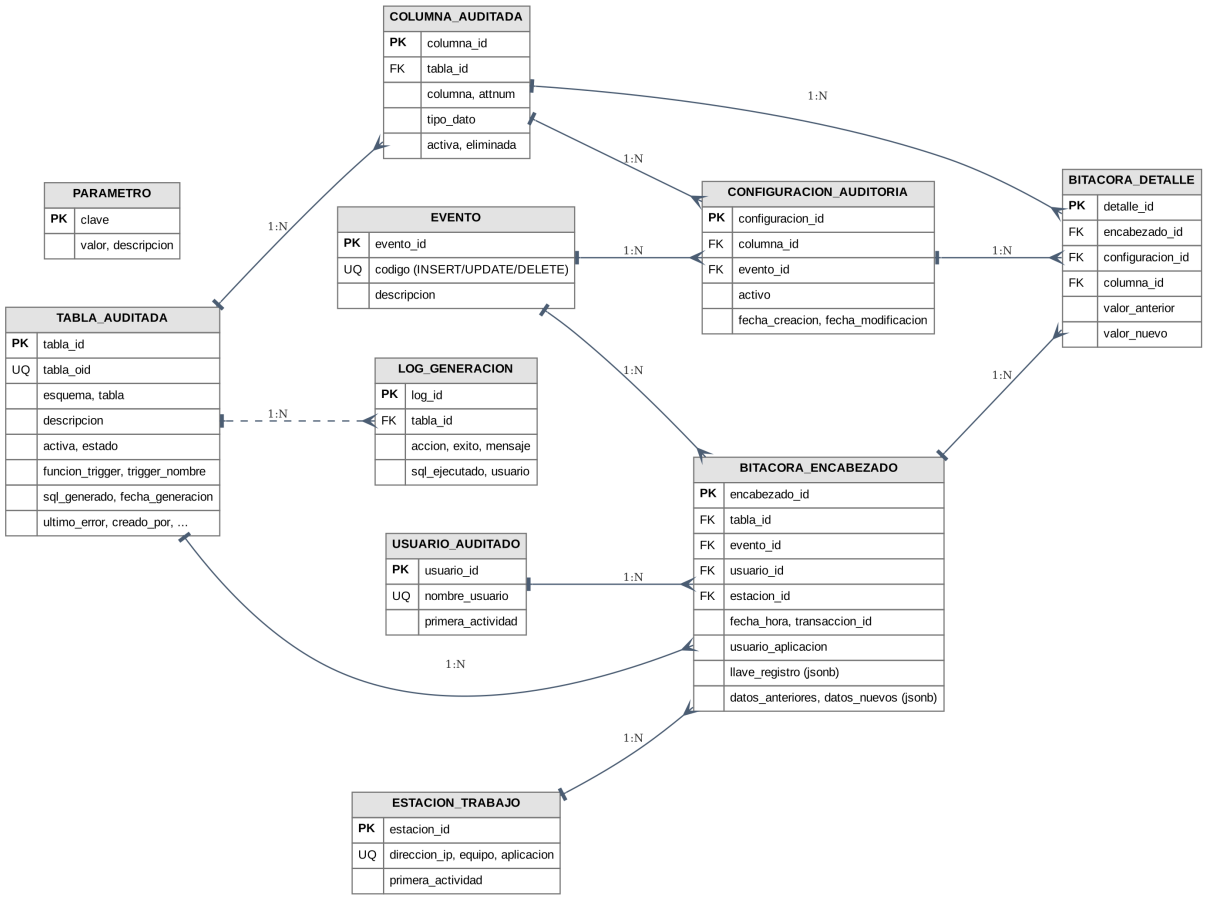


Figura 1. Diagrama entidad-relación del sistema AutoAudit

4.1 Relaciones entre entidades

Relación	Cardinalidad	Descripción
TABLA_AUDITADA – COLUMNA_AUDITADA	1 : N	Cada tabla auditada tiene varias columnas seleccionadas.
COLUMNA_AUDITADA – CONFIGURACION_AUDITORIA – EVENTO	N : M	La configuración indica qué eventos se registran ante el cambio de cada columna de cada tabla.

Relación	Cardinalidad	Descripción
TABLA_AUDITADA, EVENTO, USUARIO_AUDITADO, ESTACION_TRABAJO – BITACORA_ENCABEZADO	1 : N	Cada operación registrada referencia la tabla, el evento, el usuario responsable y la estación de trabajo.
BITACORA_ENCABEZADO – BITACORA_DETALLE	1 : N	Una operación afecta una o varias columnas.
CONFIGURACION_AUDITORIA – BITACORA_DETALLE	1 : N	Cada registro de la bitácora referencia la configuración específica que lo originó.
TABLA_AUDITADA – LOG_GENERACION	1 : N	Historial de generación de triggers y errores.

Las llaves foráneas de la bitácora usan la regla *ON DELETE RESTRICT*: una configuración con historial no puede eliminarse, solo desactivarse. Si la tabla de origen es eliminada, su configuración queda en estado HUÉRFANA y la bitácora se conserva íntegra, incluida la fila completa en formato JSON.

5. Diccionario de datos

El diccionario se obtuvo directamente del catálogo de PostgreSQL (esquema **auditoria**). Nomenclatura de llaves: PK llave primaria, FK llave foránea, UQ valor único.

5.1 EVENTO

Catálogo de operaciones DML que pueden auditarse

Columna	Tipo de dato	Nulo	Llave	Descripción
evento_id	smallint	No	PK	Identificador del evento (1 INSERT, 2 UPDATE, 3 DELETE)
codigo	character varying(10)	No	UQ	Código de la operación DML
descripcion	character varying(100)	No		Descripción legible del evento

5.2 TABLA_AUDITADA

Tablas origen configuradas para auditoría (una fila = un trigger)

Columna	Tipo de dato	Nulo	Llave	Descripción
tabla_id	integer	No	PK	Identificador de la configuración de la tabla
tabla_oid	oid	No	UQ	OID de la tabla en pg_class; identifica la tabla aunque se renombre
esquema	name	No		Esquema de la tabla origen
tabla	name	No		Nombre de la tabla origen
descripcion	character varying(250)	Sí		Motivo o descripción de la auditoría
activa	boolean	No		Interruptor maestro: si es falso no existe trigger
estado	character varying(15)	No		PENDIENTE, ACTIVO, INACTIVO, ERROR o HUERFANA (tabla origen eliminada)
funcion_trigger	name	Sí		Nombre de la función PL/pgSQL generada en el esquema auditoria
trigger_nombre	name	Sí		Nombre del trigger generado en la tabla origen (trg_autoaudit)
sql_generado	text	Sí		Último código SQL generado y aplicado
fecha_generacion	timestamp with time zone	Sí		Fecha y hora de la última generación exitosa

Columna	Tipo de dato	Nulo	Llave	Descripción
ultimo_error	text	Sí		Último error de validación o generación
creado_por	name	No		Usuario que registró la tabla
fecha_creacion	timestamp with time zone	No		Fecha de registro
modificado_por	name	Sí		Último usuario que modificó la configuración
fecha_modificacion	timestamp with time zone	Sí		Fecha de la última modificación

5.3 COLUMNA_AUDITADA

Columnas de cada tabla auditada

Columna	Tipo de dato	Nulo	Llave	Descripción
columna_id	integer	No	PK	Identificador de la columna auditada
tabla_id	integer	No	FK UQ	Tabla a la que pertenece (FK tabla_auditada)
columna	name	No		Nombre actual de la columna
attnum	smallint	No	UQ	Número de columna en pg_attribute; sobrevive a un RENAME
tipo_dato	text	No		Tipo de dato según el diccionario
activa	boolean	No		Indica si la columna participa en la auditoría
eliminada	boolean	No		La columna ya no existe en la tabla origen (DROP COLUMN)
fecha_creacion	timestamp with time zone	No		Fecha de registro

5.4 CONFIGURACION_AUDITORIA

Eventos a auditar por cada columna de cada tabla

Columna	Tipo de dato	Nulo	Llave	Descripción
configuracion_id	integer	No	PK	Identificador de la configuración columna-evento
columna_id	integer	No	FK UQ	Columna configurada (FK columna_auditada)
evento_id	smallint	No	FK UQ	Evento que se audita (FK evento)
activo	boolean	No		Si el evento se registra para esa columna
fecha_creacion	timestamp with time zone	No		Fecha de creación
fecha_modificacion	timestamp with time zone	Sí		Fecha de la última activación/desactivación

5.5 USUARIO_AUDITADO

Usuarios de base de datos detectados en la bitácora

Columna	Tipo de dato	Nulo	Llave	Descripción
usuario_id	integer	No	PK	Identificador del usuario auditado
nombre_usuario	name	No	UQ	Usuario de PostgreSQL (session_user) que realizó cambios
primera_actividad	timestamp with time zone	No		Primera vez que aparece en la bitácora

5.6 ESTACION_TRABAJO

Estaciones de trabajo desde donde se originan los cambios

Columna	Tipo de dato	Nulo	Llave	Descripción
estacion_id	integer	No	PK	Identificador de la estación
direccion_ip	character varying(64)	No	UQ	IP de la conexión (inet_client_addr), "local" si es socket, según política de privacidad
equipo	character varying(128)	No	UQ	Nombre del equipo informado por la aplicación (autoaudit.equipo)
aplicacion	character varying(128)	No	UQ	application_name de la conexión
primera_actividad	timestamp with time zone	No		Primera vez que aparece en la bitácora

5.7 BITACORA_ENCABEZADO

Bitácora: una fila por registro afectado. Conserva la fila completa aunque el origen sea eliminado

Columna	Tipo de dato	Nulo	Llave	Descripción
encabezado_id	bigint	No	PK	Identificador de la operación
fecha_hora	timestamp with time zone	No		Fecha y hora exacta del cambio
tabla_id	integer	No	FK	Tabla afectada (FK tabla_auditada)
evento_id	smallint	No	FK	Operación realizada (FK evento)
usuario_id	integer	No	FK	Usuario responsable (FK usuario_auditado)
estacion_id	integer	No	FK	Estación de origen (FK estacion_trabajo)
usuario_aplicacion	character varying(128)	Sí		Usuario de la aplicación, si la aplicación lo informa
transaccion_id	bigint	No		Identificador de la transacción (txid)
llave_registro	jsonb	Sí		Llave primaria del registro afectado en formato JSON
datos_anteriores	jsonb	Sí		Fila completa antes del cambio (UPDATE/DELETE)
datos_nuevos	jsonb	Sí		Fila completa después del cambio (INSERT/UPDATE)

5.8 BITACORA_DETALLE

Bitácora: valor anterior y nuevo por columna afectada

Columna	Tipo de dato	Nulo	Llave	Descripción
detalle_id	bigint	No	PK	Identificador del detalle
encabezado_id	bigint	No	FK	Operación a la que pertenece (FK bitacora_encabezado)
configuracion_id	integer	No	FK	Configuración columna-evento usada (FK configuracion_auditoria)
columna_id	integer	No	FK	Columna afectada (FK columna_auditada)
valor_anterior	text	Sí		Valor antes del cambio, como texto
valor_nuevo	text	Sí		Valor después del cambio, como texto

5.9 LOG_GENERACION

Registro de acciones del generador de triggers y errores

Columna	Tipo de dato	Nulo	Llave	Descripción
log_id	bigint	No	PK	Identificador del registro
fecha_hora	timestamp with time zone	No		Fecha y hora de la acción
tabla_id	integer	Sí	FK	Tabla relacionada (FK tabla_auditada)
objeto	text	Sí		Objeto afectado (función o columna)
accion	character varying(20)	No		GENERAR, ELIMINAR, VALIDAR, SINCRONIZAR, DDL o PURGAR
exito	boolean	No		Resultado de la acción
mensaje	text	Sí		Mensaje o error
sql_ejecutado	text	Sí		Código SQL ejecutado
usuario	name	No		Usuario que ejecutó la acción

5.10 PARAMETRO

Parámetros globales: privacidad de IP y retención

Columna	Tipo de dato	Nulo	Llave	Descripción
clave	character varying(50)	No	PK	Nombre del parámetro
valor	character varying(200)	No		Valor configurado
descripcion	character varying(300)	Sí		Descripción del parámetro

5.11 Vistas

Vista	Descripción
v_bitacora	Bitácora desnormalizada para consulta y reportes
v_dic_columnas	Diccionario: columnas de las tablas de usuario
v_dic_llaves	Diccionario: llaves primarias, foráneas y únicas
v_dic_tablas	Diccionario: tablas de usuario existentes
v_dic_triggers	Diccionario: triggers definidos por usuarios
v_estado_auditoria	Estado actual de la auditoría por tabla
v_registros_eliminados	
v_resumen_usuarios	

5.12 Funciones

Función	Descripción
fn_activar_tabla	Activa (genera el trigger) o desactiva (lo elimina) la auditoría de una tabla.
fn_configurar_columna	Valida la columna y guarda los eventos (INSERT/UPDATE/DELETE) a auditar.
fn_configurar_tabla	Configura varias columnas con los mismos eventos y genera el trigger.
fn_construir_sql_trigger	Construye el código del trigger sin ejecutarlo (vista previa).
fn_consultar_bitacora	Consulta parametrizada de la bitácora con filtros opcionales.
fn_crear_encabezado	Inserta el encabezado de bitácora con usuario y estación de trabajo.
fn_dic_llave_primaria	Obtiene las columnas de la llave primaria de una tabla.
fn_dic_oid_tabla	Obtiene el identificador interno (OID) de una tabla.
fn_eliminar_configuracion	Elimina la configuración de una tabla sin registros en bitácora.
fn_eliminar_trigger	Elimina el trigger y la función generada de una tabla.
fn_eliminar_trigger_interno	Implementación interna de la eliminación.
fn_es_admin	Indica si el usuario de la sesión es administrador de auditoría.
fn_estadisticas	Indicadores para el panel principal de la aplicación.
fn_evt_ddl_end	Event trigger: sincroniza y regenera el trigger tras un ALTER TABLE.
fn_evt_sql_drop	Event trigger: detecta la eliminación de tablas o columnas auditadas.
fn_generar_trigger	Valida, construye y ejecuta el trigger; registra el resultado.
fn_generar_trigger_interno	Implementación interna del generador (no se otorga a usuarios).
fn_historial_columna	Todos los cambios realizados en una columna específica.
fn_historial_registro	Línea de tiempo de un registro a partir de su llave primaria.
fn_historial_usuario	Historial completo de modificaciones de un usuario.
fn_ip_cliente	Obtiene la IP de origen aplicando la política de privacidad.
fn_proteger_bitacora	Impide modificar o borrar registros de la bitácora.
fn_purgar_bitacora	Depura la bitácora según el período de retención (solo superusuario).
fn_regenerar_todos	Regenera los triggers de todas las tablas activas.
fn_registrar_tabla	Valida la tabla en el diccionario y la registra para auditoría.
fn_set_parametro	Modifica un parámetro global del sistema.
fn_sincronizar_tabla	Actualiza nombres renombrados y marca columnas eliminadas.
fn_validar_configuracion	Compara la configuración contra el diccionario (errores y avisos).

Función	Descripción
fn_verificar_admin	Detiene la operación si el usuario no es administrador.

6. Generación dinámica de triggers

La función **fn_generar_trigger** valida la configuración contra el diccionario, construye el código con **fn_construir_sql_trigger** y lo ejecuta. Si ocurre un error, la operación se revierte, la tabla queda en estado ERROR y el detalle se guarda en LOG_GENERACION, sin afectar el trigger anterior. Las características del código generado son:

- Una función PL/pgSQL exclusiva para cada tabla de origen, tal como lo exige el proyecto.
- Trigger de tipo *AFTER ... FOR EACH ROW* que solo se dispara en los eventos configurados; en UPDATE se limita a las columnas auditadas (*UPDATE OF columnas*).
- Las columnas y eventos quedan escritos en el código, por lo que no se consulta la configuración en cada fila, reduciendo el impacto en el rendimiento.
- En UPDATE solo se registran los valores que realmente cambiaron (comparación IS DISTINCT FROM).
- En DELETE se preserva la fila completa, de modo que la información permanece disponible aunque el registro original ya no exista.
- Event triggers mantienen la coherencia ante RENAME, DROP COLUMN y DROP TABLE, regenerando el trigger automáticamente.

6.1 Ejemplo de código generado (tabla tienda.productos)

```
-- ============================================================================
-- Código generado automáticamente por AutoAudit. NO EDITAR A MANO.
-- Tabla origen : tienda.productos   (tabla_id = 2)
-- Generado     : 2026-11-03 22:54:00 por postgres
-- ============================================================================
CREATE OR REPLACE FUNCTION auditoria.fn_aud_2_tienda_productos()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = pg_catalog, auditoria
AS $autoaudit$
DECLARE
    v_enc bigint;
BEGIN
    IF TG_OP = 'INSERT' THEN
        v_enc := auditoria.fn_crear_encabezado(2, 1::smallint, NULL, to_jsonb(NEW),
            jsonb_build_object('producto_id', NEW.producto_id));
        INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id,
            valor_anterior, valor_nuevo)
        VALUES (v_enc, 17, 6, NULL, NEW.nombre::text),
               (v_enc, 20, 7, NULL, NEW.precio::text);
        RETURN NULL;
    END IF;
    IF TG_OP = 'UPDATE' THEN
        IF OLD.nombre IS DISTINCT FROM NEW.nombre THEN
            v_enc := COALESCE(v_enc, auditoria.fn_crear_encabezado(2, 2::smallint, to_jsonb(OLD),
                to_jsonb(NEW), jsonb_build_object('producto_id', NEW.producto_id)));
            INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id,
                valor_anterior, valor_nuevo)
            VALUES (v_enc, 16, 6, OLD.nombre::text, NEW.nombre::text);
        END IF;
        IF OLD.precio IS DISTINCT FROM NEW.precio THEN
            v_enc := COALESCE(v_enc, auditoria.fn_crear_encabezado(2, 2::smallint, to_jsonb(OLD),
                to_jsonb(NEW), jsonb_build_object('producto_id', NEW.producto_id)));
            INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id,
                valor_anterior, valor_nuevo)
            VALUES (v_enc, 19, 7, OLD.precio::text, NEW.precio::text);
        END IF;
        IF OLD.existencia IS DISTINCT FROM NEW.existencia THEN
            v_enc := COALESCE(v_enc, auditoria.fn_crear_encabezado(2, 2::smallint, to_jsonb(OLD),
                to_jsonb(NEW), jsonb_build_object('producto_id', NEW.producto_id)));
            INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id,
                valor_anterior, valor_nuevo)
            VALUES (v_enc, 22, 8, OLD.existencia::text, NEW.existencia::text);
        END IF;
        RETURN NULL;
    END IF;
    IF TG_OP = 'DELETE' THEN
        v_enc := auditoria.fn_crear_encabezado(2, 3::smallint, to_jsonb(OLD), NULL,
            jsonb_build_object('producto_id', OLD.producto_id));
        INSERT INTO auditoria.bitacora_detalle (encabezado_id, configuracion_id, columna_id,
            valor_anterior, valor_nuevo)
        VALUES (v_enc, 18, 6, OLD.nombre::text, NULL),
               (v_enc, 21, 7, OLD.precio::text, NULL);
        RETURN NULL;
```

```
    END IF;
    RETURN NULL;
END;
$autoaudit$;

ALTER FUNCTION auditoria.fn_aud_2_tienda_productos() OWNER TO autoaudit_owner;
REVOKE ALL ON FUNCTION auditoria.fn_aud_2_tienda_productos() FROM PUBLIC;
COMMENT ON FUNCTION auditoria.fn_aud_2_tienda_productos() IS 'AutoAudit: trigger de auditoría de
    tienda.productos';

CREATE OR REPLACE TRIGGER trg_autoaudit
    AFTER INSERT OR UPDATE OF nombre, precio, existencia OR DELETE
    ON tienda.productos
    FOR EACH ROW
    EXECUTE FUNCTION auditoria.fn_aud_2_tienda_productos();
```

7. Manual de configuración y activación de auditoría

Solo los usuarios del rol **autoaudit_admin** pueden configurar la auditoría. El usuario de demostración es **admin_auditoria** (contraseña Admin123!).

7.1 Desde la interfaz web

1. Iniciar la aplicación con **iniciar_windows.bat** (Windows) o **bash run.sh** (Ubuntu) y abrir http://localhost:8000 en el navegador.
2. Ingresar como admin_auditoria y abrir el módulo **Configuración**. A la izquierda se muestran las tablas leídas del diccionario de datos con su estado de auditoría.
3. Seleccionar la tabla. Se listan sus columnas con tipo de dato, llaves y tres casillas: INSERT, UPDATE y DELETE. Al hacer clic en el título de un evento se marcan todas las columnas.
4. Pulsar **Guardar y generar trigger**. El sistema valida, genera y ejecuta el código, y muestra el SQL resultante. **Guardar sin aplicar** solo almacena la configuración.
5. **Desactivar** elimina el trigger conservando la configuración y la bitácora; **Activar** lo vuelve a generar; **Regenerar** lo reconstruye.
6. Los cambios quedan registrados y pueden consultarse en los módulos **Bitácora**, **Por usuario** y **Por columna**, con exportación a CSV, Excel y PDF.

7.2 Desde SQL

```
-- Consultar el diccionario de datos
SELECT * FROM auditoria.v_dic_tablas;
SELECT * FROM auditoria.v_dic_columnas WHERE esquema = 'tienda' AND tabla = 'productos';

-- Configurar columnas (INSERT, UPDATE, DELETE) y generar el trigger
SELECT auditoria.fn_configurar_columna('tienda', 'productos', 'precio',     true,  true, true);
SELECT auditoria.fn_configurar_columna('tienda', 'productos', 'existencia', false, true, false);
SELECT auditoria.fn_generar_trigger(tabla_id) FROM auditoria.tabla_auditada
 WHERE esquema = 'tienda' AND tabla = 'productos' AND estado <> 'HUERFANA';

-- Activar / desactivar y revisar el estado
SELECT auditoria.fn_activar_tabla(1, false);
SELECT auditoria.fn_activar_tabla(1, true);
SELECT * FROM auditoria.v_estado_auditoria;

-- Consultas históricas
SELECT * FROM auditoria.fn_historial_usuario('vendedor');
SELECT * FROM auditoria.fn_historial_columna('tienda', 'productos', 'precio');
```

7.3 Estados de una tabla

Estado	Significado
PENDIENTE	Configuración guardada o modificada; falta generar el trigger.

Estado	Significado
ACTIVO	Trigger generado y vigente.
INACTIVO	Auditoría desactivada; se conservan la configuración y la bitácora.
ERROR	La validación o la generación falló; el detalle está en LOG_GENERACION.
HUERFANA	La tabla de origen fue eliminada; se conserva su historial.

8. Procedimiento de instalación y despliegue

8.1 Requisitos

- PostgreSQL 18 (o versión 14 en adelante).
- Python 3.10 o superior (en Windows, con la opción “Add python.exe to PATH”).
- Sistema operativo Ubuntu 26.04 LTS o Windows 10/11.

8.2 Instalación en Windows

1. Descargar o clonar el repositorio y ubicarlo en una carpeta fuera de OneDrive, por ejemplo C:\autoaudit.
2. Ejecutar **instalar_windows.bat** e ingresar la contraseña del usuario postgres. El instalador crea la base de datos autoaudit_db, el esquema de auditoría, el generador, la seguridad, la base de ejemplo y el entorno de Python.
3. Ejecutar **iniciar_windows.bat** y abrir http://localhost:8000. La ventana debe permanecer abierta mientras se use la aplicación.
4. Ejecutar **pruebas\ejecutar_pruebas_windows.bat** para validar la instalación.

8.3 Instalación en Ubuntu 26.04 LTS

```
sudo bash install.sh                    # PostgreSQL, base de datos, demo y entorno Python
bash run.sh                             # inicia la aplicación en http://localhost:8000
sudo bash pruebas/ejecutar_pruebas.sh   # pruebas de validación
```

8.4 Orden de los scripts de base de datos

Script	Contenido
00_roles_y_bd.sql	Roles de seguridad, usuarios de demostración y base de datos.
01_esquema.sql	Tablas del sistema, índices, catálogo de eventos y parámetros.
02_diccionario.sql	Vistas de consulta al diccionario de datos del DBMS.
03_funciones_config.sql	Módulo de configuración con validaciones.
04_generador_triggers.sql	Generador dinámico de triggers y registro en bitácora.
05_consultas.sql	Consultas parametrizadas de historial y estadísticas.
06_seguridad.sql	Permisos y protección de la bitácora.
07_eventos_ddl.sql	Gestión de dependencias ante cambios de estructura.
08_datos_demo.sql	Base de datos de ejemplo (esquema tienda).
09_comentarios.sql	Descripciones para el diccionario de datos.

8.5 Usuarios de demostración

Usuario	Contraseña	Rol
admin_auditoria	Admin123!	Configura la auditoría y consulta la bitácora
auditor	Auditor123!	Solo consulta la bitácora
vendedor	Vendedor123!	Usuario operativo (clientes y pedidos)
bodeguero	Bodega123!	Usuario operativo (productos)

9. Consideraciones de seguridad

Requisito	Implementación
Solo administradores configuran la auditoría	Las funciones de configuración verifican la pertenencia al rol autoaudit_admin; los administradores no modifican las tablas de forma directa.
Sin modificación directa de la bitácora	Solo el rol técnico autoaudit_owner, dueño de los triggers generados, puede insertar. Un trigger de protección impide UPDATE, DELETE y TRUNCATE, incluso al superusuario.
Usuario responsable confiable	Se registra session_user, que no cambia con SET ROLE ni dentro de funciones SECURITY DEFINER.
Privacidad de la dirección IP	Parámetro ip_modo: completa, anonimizada o ninguna. La IP se toma de la conexión y el cliente no puede falsificarla.
Prevención de inyección SQL	El generador usa format() con identificadores y literales escapados; la aplicación usa consultas parametrizadas.
Retención de datos	Parámetro dias_retencion y función de depuración restringida al superusuario.

10. Pruebas de validación

Las pruebas se ejecutaron con el script **pruebas\ejecutar_pruebas_windows.bat** sobre la instalación del grupo. El resultado fue de **39 pruebas aprobadas y 0 fallidas**.

10.1 Casos de prueba

No.	Caso de prueba	Resultado
1	INSERT genera encabezado en bitácora	Aprobada
2	INSERT registra la llave primaria del registro	Aprobada
3	INSERT guarda la fila completa nueva (datos_nuevos)	Aprobada
4	INSERT registra exactamente las 3 columnas configuradas	Aprobada
5	INSERT guarda valor nuevo de saldo = 1500.00	Aprobada
6	INSERT no registra columnas no configuradas (notas)	Aprobada
7	INSERT registra el usuario responsable (vendedor)	Aprobada
8	INSERT registra estación de trabajo (equipo y aplicación)	Aprobada
9	INSERT registra fecha/hora y transacción	Aprobada
10	UPDATE: solo 2 de 4 sentencias generan bitácora (cambios reales en columnas auditadas)	Aprobada
11	UPDATE captura valor anterior (1500.00) y nuevo (2500.75) de saldo	Aprobada
12	UPDATE registra usuario (bodeguero) y estación (PC-BODEGA-02)	Aprobada
13	UPDATE de 2 columnas genera 1 encabezado con 2 detalles	Aprobada
14	UPDATE captura estado ACTIVA -> BLOQUEADA	Aprobada
15	UPDATE de columna no auditada (notas) no se registra	Aprobada
16	UPDATE guarda la fila completa anterior (datos_anteriores)	Aprobada
17	DELETE: el registro ya no existe en la tabla origen	Aprobada
18	DELETE genera encabezado en bitácora	Aprobada
19	DELETE preserva la fila COMPLETA eliminada (incluye columnas no auditadas)	Aprobada
20	DELETE no tiene datos nuevos	Aprobada
21	DELETE guarda valor anterior de las 3 columnas auditadas	Aprobada
22	Historial del registro eliminado sigue disponible (INSERT + DELETE)	Aprobada
23	Vista v_registros_eliminados muestra el registro	Aprobada
24	Usuario sin privilegios no puede configurar	Aprobada
25	Escritura directa en bitácora denegada (usuario común)	Aprobada
26	Administrador tampoco puede borrar la bitácora	Aprobada
27	Bitácora inmutable incluso para superusuario	Aprobada

No.	Caso de prueba	Resultado
28	Valida existencia de tabla	Aprobada
29	Valida existencia de columna	Aprobada
30	RENAME COLUMN: la configuración se sincroniza automáticamente	Aprobada
31	DROP COLUMN de columna auditada es bloqueado por dependencia	Aprobada
32	DROP COLUMN: se marca la columna y el trigger se regenera sin romper DML	Aprobada
33	Tabla de configuración refleja el estado real del trigger	Aprobada
34	Consulta por usuario retorna su historial	Aprobada
35	Consulta por columna retorna solo cambios de esa columna	Aprobada
36	Consulta filtrada por operación funciona	Aprobada
37	Desactivar elimina el trigger	Aprobada
38	Reactivar vuelve a generar el trigger	Aprobada
39	Rendimiento: todas las operaciones masivas quedaron auditadas (183.000)	Aprobada

10.2 Rendimiento con volumen creciente de registros

Se midió el tiempo de operaciones masivas sobre dos tablas idénticas, una sin auditoría y otra con auditoría de tres columnas, para 1,000, 10,000 y 50,000 registros. El sobrecosto por fila es el tiempo adicional que agrega la auditoría a cada registro procesado.

Registros	Operación	Sin auditoría (ms)	Con auditoría (ms)	Sobrecosto por fila (ms)
1,000	INSERT	3.9	110.9	0.1070
1,000	UPDATE	2.7	83.0	0.0803
1,000	DELETE	0.3	97.7	0.0974
10,000	INSERT	28.3	1,062.3	0.1034
10,000	UPDATE	25.2	889.0	0.0864
10,000	DELETE	3.1	2,354.7	0.2352
50,000	INSERT	508.8	17,106.3	0.3320
50,000	UPDATE	482.2	18,786.3	0.3661
50,000	DELETE	76.5	19,286.2	0.3842

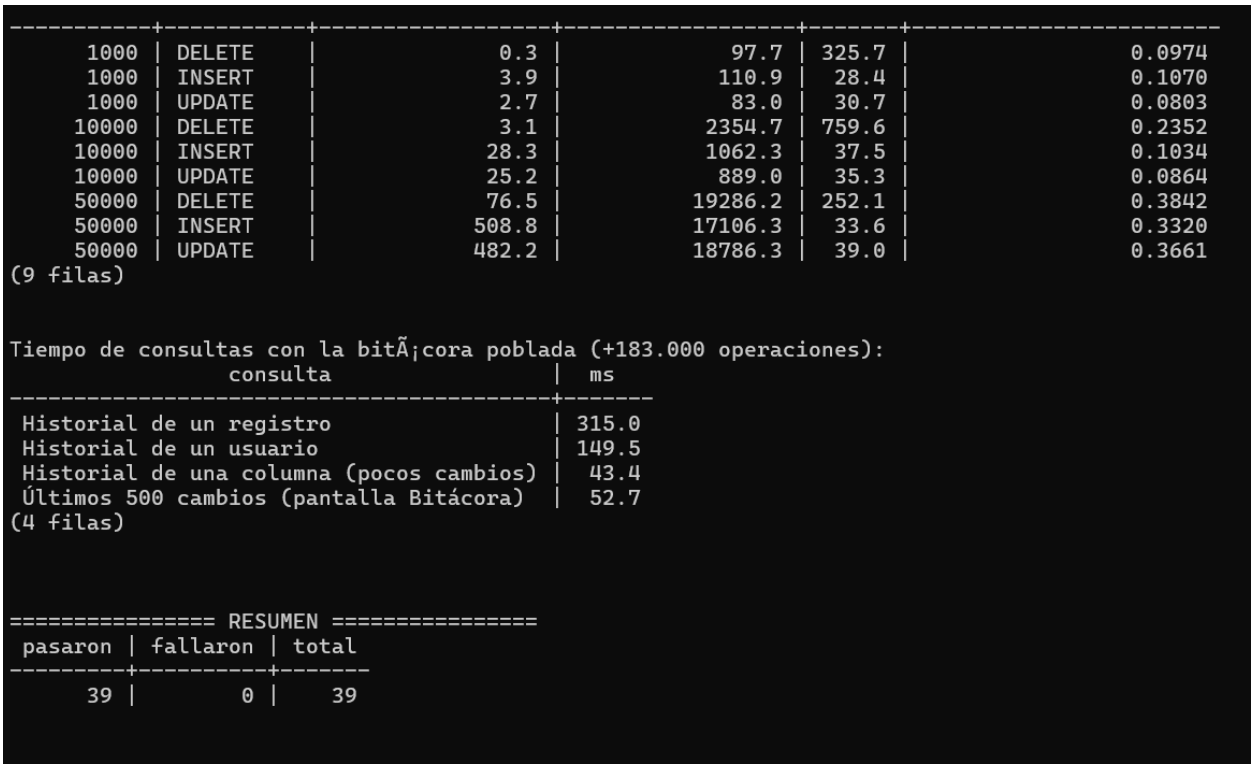
El sobrecosto se mantiene por debajo de 0.4 milisegundos por fila incluso con 50,000 registros. En el uso normal, donde cada operación afecta a uno o pocos registros, el impacto es imperceptible para el usuario. En operaciones masivas el tiempo total aumenta porque cada fila genera sus propios registros de bitácora, comportamiento

esperado en cualquier mecanismo de auditoría por trigger.

10.3 Tiempo de consultas con la bitácora poblada

Con más de 183,000 operaciones registradas en la bitácora se obtuvieron los siguientes tiempos:

Consulta	Tiempo (ms)
Historial de un registro	315.0
Historial de un usuario	149.5
Historial de una columna	43.4
Últimos 500 cambios (pantalla Bitácora)	52.7



```
-----------+-----------+------------------+------------------+-------+-----------------------
      1000 | DELETE    |              0.3 |             97.7 | 325.7 |                0.0974
      1000 | INSERT    |              3.9 |            110.9 |  28.4 |                0.1070
      1000 | UPDATE    |              2.7 |             83.0 |  30.7 |                0.0803
     10000 | DELETE    |              3.1 |           2354.7 | 759.6 |                0.2352
     10000 | INSERT    |             28.3 |           1062.3 |  37.5 |                0.1034
     10000 | UPDATE    |             25.2 |            889.0 |  35.3 |                0.0864
     50000 | DELETE    |             76.5 |          19286.2 | 252.1 |                0.3842
     50000 | INSERT    |            508.8 |          17106.3 |  33.6 |                0.3320
     50000 | UPDATE    |            482.2 |          18786.3 |  39.0 |                0.3661
(9 filas)


Tiempo de consultas con la bitÃ¡cora poblada (+183.000 operaciones):
                consulta                  |  ms
------------------------------------------+-------
 Historial de un registro                 | 315.0
 Historial de un usuario                  | 149.5
 Historial de una columna (pocos cambios) |  43.4
 Últimos 500 cambios (pantalla Bitácora)  |  52.7
(4 filas)



================ RESUMEN ================
 pasaron | fallaron | total
---------+----------+-------
      39 |        0 |    39
```

Figura 2. Evidencia de la ejecución de las pruebas de validación (39 aprobadas, 0 fallidas)

11. Conclusiones

1. La generación dinámica de triggers a partir del diccionario de datos permite auditar cualquier tabla sin escribir código manual, reduciendo errores y tiempo de implementación.
2. La separación de la bitácora en encabezado y detalle, junto con la conservación de la fila completa, garantiza que la información auditada permanezca disponible

aunque el registro original sea eliminado.

3. Los controles de seguridad basados en roles y la protección de la bitácora aseguran que los registros de auditoría no puedan ser alterados por los usuarios.
4. Las pruebas de validación confirmaron el registro correcto de inserciones, actualizaciones y eliminaciones, y un impacto en el rendimiento bajo y lineal respecto al volumen de datos.

12. Referencias

- PostgreSQL Global Development Group. *PostgreSQL 18 Documentation*: Triggers, Event Triggers, PL/pgSQL y System Catalogs. https://www.postgresql.org/docs/
- FastAPI. *Documentación oficial*. https://fastapi.tiangolo.com/
- Psycopg. *Psycopg 3 documentation*. https://www.psycopg.org/psycopg3/docs/